

Publishing API – Developer Manual

Version: 1.0.0

OpenAPI Spec Version: 3.1.0

Base URL:

`https://api.publishinghub.example/v1`

A hypothetical REST API for managing a publishing catalog: books, authors, and orders. Built to demonstrate a complete OpenAPI-based manual structure.

1. Overview

The Publishing API lets client applications create, retrieve, update,

and delete records for books, authors, and orders. It follows REST conventions, uses JSON for all request and response bodies, and is versioned via the URL path (/v1).

Property	Value
Protocol	HTTPS only
Data format	application/json
Auth	Bearer token (API key)
Rate limit	100 requests / minute / key
Pagination	Cursor-based

2. Authentication

All requests require an API key passed as a Bearer token.

```
Authorization: Bearer  
<YOUR_API_KEY>
```

OpenAPI security scheme:

```
components:  
  securitySchemes:  
    BearerAuth:  
      type: http  
      scheme: bearer  
      bearerFormat: API-KEY  
security:  
  - BearerAuth: []
```

Requests without a valid key return

401 Unauthorized:

```
{  
  "error": "unauthorized",  
  "message": "Missing or
```

```
invalid API key."
}
```

3. Common Conventions

- **Dates:** ISO 8601 (2026-07-27T14:00:00Z)
- **IDs:** UUID v4 strings
- **Pagination params:** `limit` (default 20, max 100), `cursor` (opaque string)
- **Errors:** consistent envelope —

```
{
  "error": "<error_code>",
  "message": "<human readable>",
  "details": {}
}
```

HTTP Status	Meaning
200	Success
201	Created
204	Deleted, no content
400	Validation error
401	Unauthorized
404	Not found
409	Conflict (e.g., duplicate ISBN)
429	Rate limited
500	Server error

4. Endpoints

4.1 Books

GET /books

List books.

Query params:

Name	Type	Description
limit	integer	Page size (max 100)
cursor	string	Pagination cursor
author_id	uuid	Filter by author
genre	string	Filter by genre

Response 200:

```
{
  "data": [
    {
      "id": "b1f2c3d4-0000-4000-8000-000000000001",
      "title": "The Garden of
```

```
Daisy",
  "author_id": "a1f2c3d4-
0000-4000-8000-0000000000010",
  "isbn": "978-1-000000-
01-1",
  "genre": "children",
  "published_at": "2025-
03-01",
  "price_usd": 9.99
}
],
"next_cursor":
"eyJvZmZzZXQiOi0jIw=="
}
```

POST /books

Create a book.

Request body:

```
{
  "title": "The Garden of
Daisy",
```

```
"author_id": "a1f2c3d4-0000-4000-8000-000000000010",  
  "isbn": "978-1-000000-01-1",  
  "genre": "children",  
  "published_at": "2025-03-01",  
  "price_usd": 9.99  
}
```

Response 201 : returns the created book object (same shape as above, with `id`).

Errors: 400 invalid ISBN format, 409 ISBN already exists.

GET /books/{book_id}

Retrieve a single book. 404 if not found.

PATCH /books/{book_id}

Partial update. Accepts any subset of the book fields. Returns 200 with the updated object.

DELETE /books/{book_id}

Deletes a book. Returns 204 .

4.2 Authors

GET /authors

List authors. Same pagination pattern as Books.

POST /authors

```
{
  "name": "Mary Lou Author",
  "bio": "Independent author
and publisher.",
  "website":
```

```
"https://example.com"  
}
```

Returns `201` with the created author, including `id`.

GET /authors/{author_id}

Retrieve one author, including a `books` summary array of `{id, title}`.

PATCH /authors/{author_id} /
DELETE /authors/{author_id}

Standard partial update / delete, same semantics as Books.

4.3 Orders

POST /orders

Create an order for one or more books.

```
{
  "customer_email":
  "reader@example.com",
  "items": [
    { "book_id": "b1f2c3d4-
0000-4000-8000-0000000000001",
    "quantity": 2 }
  ]
}
```

Response 201 :

```
{
  "id": "o1f2c3d4-0000-4000-
8000-0000000000100",
  "status": "pending",
  "total_usd": 19.98,
  "items": [
    { "book_id": "b1f2c3d4-
0000-4000-8000-0000000000001",
```

```
"quantity": 2,  
"unit_price_usd": 9.99 }  
],  
"created_at": "2026-07-  
27T14:00:00Z"  
}
```

GET /orders/{order_id}

Retrieve order status: pending,
fulfilled, cancelled.

POST /orders/{order_id}/cancel

Cancels a pending order. Returns 200
with updated status. 409 if already
fulfilled.

5. Schemas (OpenAPI Components)

```
components:
  schemas:
    Book:
      type: object
      required: [title,
author_id, isbn]
      properties:
        id: { type: string,
format: uuid, readOnly: true }
        title: { type: string,
maxLength: 200 }
        author_id: { type:
string, format: uuid }
        isbn: { type: string,
pattern: '^\\d{3}-\\d-\\d{6}-
\\d{2}-\\d$' }
        genre: { type: string
}
        published_at: { type:
string, format: date }
        price_usd: { type:
number, minimum: 0 }
```

```
Author:
  type: object
  required: [name]
  properties:
    id: { type: string,
format: uuid, readOnly: true }
    name: { type: string }
    bio: { type: string }
    website: { type:
string, format: uri }
```

```
Order:
  type: object
  properties:
    id: { type: string,
format: uuid, readOnly: true }
    customer_email: {
type: string, format: email }
    status: { type:
string, enum: [pending,
fulfilled, cancelled] }
    total_usd: { type:
number }
```

```
    items:
      type: array
      items:
        type: object
        properties:
          book_id: { type:
string, format: uuid }
          quantity: {
type: integer, minimum: 1 }
          unit_price_usd:
{ type: number }
```

```
    Error:
      type: object
      properties:
        error: { type: string
}
        message: { type:
string }
        details: { type:
object }
```

6. Rate Limiting

- 100 requests/minute per API key.
- Headers returned on every response:

```
X-RateLimit-Limit: 100  
X-RateLimit-Remaining: 87  
X-RateLimit-Reset: 1753624860
```

- Exceeding the limit returns 429 with Retry-After (seconds).
-

7. Example Requests (curl)

List books by genre:

```
curl -H "Authorization: Bearer  
$API_KEY" \
```



```
"https://api.publishinghub.example/v1/books?genre=children&limit=10"
```

Create an author:

```
curl -X POST  
"https://api.publishinghub.example/v1/authors" \  
  -H "Authorization: Bearer $API_KEY" \  
  -H "Content-Type: application/json" \  
  -d '{"name": "Mary Lou Author", "bio": "Independent author and publisher."}'
```

Place an order:

```
curl -X POST  
"https://api.publishinghub.example/v1/orders" \  
  -H "Authorization: Bearer $API_KEY" \  
  -d '{"book_id": "1234567890", "quantity": 1}'
```

```
-H "Content-Type:
application/json" \
-d
'{"customer_email":"reader@exa
mple.com","items":
[{"book_id":"b1f2c3d4-0000-
4000-8000-
0000000000001","quantity":1}]}'
```

8. Versioning & Deprecation

- URL-based versioning (/v1 , /v2 , ...).
- Deprecated fields marked `deprecated: true` in the schema and announced 90 days before removal.
- Breaking changes always ship under a new version path; non-

breaking additions (new optional fields) may land in the current version.

9. Appendix: Full Minimal OpenAPI Skeleton

```
openapi: 3.1.0
info:
  title: Publishing API
  version: 1.0.0
servers:
  - url:
https://api.publishinghub.example/v1
paths:
  /books:
    get:
      summary: List books
      responses:
        '200':
```

```
        description: OK
    post:
        summary: Create a book
        requestBody:
            required: true
            content:
                application/json:
                    schema:
                        $ref:
'#/components/schemas/Book'
        responses:
            '201':
                description: Created
/blogs/{book_id}:
    get:
        summary: Get a book
        parameters:
            - name: book_id
              in: path
              required: true
              schema: { type:
string, format: uuid }
        responses:
```

```
'200':  
  description: OK  
'404':  
  description: Not  
found
```

This manual documents a hypothetical API for illustrative purposes. Adapt endpoint names, schemas, and auth scheme to match a real implementation before publishing.